

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT 2 - MONGODB DATABASE MANAGEMENT

Student Name: B. Manoj

Course Code: 23CM4121

Roll No.: A24126552072

Date: 04-10-2026

TITLE: MongoDB Student Information Management System

1. AIM

To develop a Node.js program to store, retrieve, update, and manage student information in MongoDB using the database **collegeDB** and collection **students**.

2. OBJECTIVES

- To create database 'collegeDB' and collection 'students'.
 - To insert, find, update, and delete student records.
 - To perform conditional queries based on branch, marks, and year.
 - To sort student records in descending order of marks.
 - To create an index on rollNo and analyze search performance.
 - To execute real-time extension queries (top scorers, low scorers, branch filtering).
-

3. THEORY

MongoDB: MongoDB is a document-oriented NoSQL database that stores data in JSON-like BSON documents inside collections.

Document Structure: Each record in the students collection contains fields: rollNo, name, branch, year, marks, and email.

Indexing: By default, queries without an index scan the entire collection (COLLSCAN). Creating an index on 'rollNo' uses an index scan (IXSCAN) which allows faster direct searches.

4. PROCEDURE

Step 1: Connect to MongoDB using MongoClient.

Step 2: Select database 'collegeDB' and collection 'students'.

Step 3: Insert initial student records with insertMany().

Step 4: Query students using find() and findOne() with branch and marks conditions.

Step 5: Update marks and email using updateOne() with \$set.

Step 6: Delete a student record using deleteOne().

Step 7: Sort records in descending order of marks using sort({ marks: -1 }).

Step 8: Create an index on rollNo with createIndex() and examine execution stats.

Step 9: Execute real-time extension queries and close connection.

5. PROGRAM CODE (student_management.js)

```
const { MongoClient } = require('mongodb');

// Connection URL and Database
const url = 'mongodb://127.0.0.1:27017';
const client = new MongoClient(url);
const dbName = 'collegeDB';

async function main() {
  try {
    await client.connect();
    console.log('Connected to MongoDB successfully.');
```

const db = client.db(dbName);
const students = db.collection('students');

// 1. Insert at least 5 to 8 student records
const studentData = [
 { rollNo: "23CM001", name: "Ravi Kumar", branch: "CSE-AIML", year: 3, marks: 85, email: "ravi@example.com" },
 { rollNo: "23CM002", name: "Priya Sharma", branch: "CSE", year: 3, marks: 92, email: "priya@example.com" },
 { rollNo: "23CM003", name: "Ananya Verma", branch: "IT", year: 2, marks: 78, email: "ananya@example.com" },
 { rollNo: "23CM004", name: "Kiran Patel", branch: "ECE", year: 4, marks: 45, email: "kiran@example.com" },
 { rollNo: "23CM005", name: "Suresh Reddy", branch: "CSE-AIML", year: 3, marks: 68, email: "suresh@example.com" },
 { rollNo: "23CM006", name: "Sneha Nair", branch: "CSE", year: 2, marks: 88, email: "sneha@example.com" },
 { rollNo: "23CM007", name: "Ramesh Rao", branch: "CSE", year: 3, marks: 95, email: "ramesh@example.com" }
];

await students.insertMany(studentData);
console.log('Students inserted successfully.');

// 2. Display all students
console.log('\n--- All Students ---');
console.log(await students.find().toArray());

// 3. Display students belonging to CSE-AIML
console.log('\n--- CSE-AIML Branch Students ---');
console.log(await students.find({ branch: "CSE-AIML" }).toArray());

// 4. Display students who scored more than 75 marks
console.log('\n--- Students with Marks > 75 ---');
console.log(await students.find({ marks: { \$gt: 75 } }).toArray());

// 5. Search for a student using rollNo
console.log('\n--- Search Student by rollNo (23CM001) ---');
console.log(await students.findOne({ rollNo: "23CM001" }));

// 6. Search students based on condition: year = 3 and marks >= 80
console.log('\n--- Year 3 Students with Marks >= 80 ---');
console.log(await students.find({ year: 3, marks: { \$gte: 80 } }).toArray());

// 7. Update marks of a particular student
await students.updateOne({ rollNo: "23CM001" }, { \$set: { marks: 92 } });
console.log('\nUpdated marks for 23CM001 to 92');

// 8. Update another field (email)
await students.updateOne({ rollNo: "23CM003" }, { \$set: { email: "ananya.official@example.com" } });
console.log('Updated email for 23CM003');

// 9. Delete a student record using rollNo
await students.deleteOne({ rollNo: "23CM005" });
console.log('\nDeleted student record for 23CM005');

// 10. Display students in descending order of marks
console.log('\n--- Students Sorted by Marks (Descending) ---');

```
console.log(await students.find().sort({ marks: -1 }).toArray());

// 11. Create an index on rollNo
await students.createIndex({ rollNo: 1 }, { unique: true });
console.log('\nCreated unique index on rollNo');

// 12. Demonstrate index performance (explain executionStats)
const stats = await students.find({ rollNo: "23CM007" }).explain("executionStats");
console.log('\nIndex Search Stage:', stats.queryPlanner.winningPlan.stage);

// 13. Real-Time Extension Queries
console.log('\n--- Real-Time Extension Queries ---');
console.log('Marks > 80:', await students.find({ marks: { $gt: 80 } }).toArray());
console.log('Marks < 50:', await students.find({ marks: { $lt: 50 } }).toArray());
console.log('Highest Scorer:', await students.find().sort({ marks: -1 }).limit(1).toArray());
console.log('IT Branch:', await students.find({ branch: "IT" }).toArray());
console.log('Sorted Ascending:', await students.find().sort({ marks: 1 }).toArray());

} catch (err) {
  console.error('Database Error:', err.message);
} finally {
  await client.close();
  console.log('\nConnection closed.');
```

```
}
}

main();
```

6. CODE EXPLANATION

Method / Query	Description
students.insertMany()	Inserts an array of student documents into the collection.
students.find({ branch: 'CSE-AIML' })	Finds and returns documents matching the specified branch.
{ marks: { \$gt: 75 } }	Filters students scoring more than 75 marks using \$gt operator.
students.findOne({ rollNo })	Searches for a single student matching the given rollNo.
students.updateOne({ rollNo }, { \$set })	Updates specific fields like marks or email for a matching student.
students.deleteOne({ rollNo })	Deletes the student record matching the rollNo.
sort({ marks: -1 })	Sorts records in descending order of marks.
students.createIndex({ rollNo: 1 })	Creates an index on rollNo to speed up search queries.

7. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Insert Students	Insert 7 student records	Records inserted with _id assigned	Pass
TC-02	Branch Filter	Query { branch: 'CSE-AIML' }	Returns 2 students in CSE-AIML	Pass
TC-03	Index Query	Search indexed rollNo	Executed via IXSCAN stage	Pass
TC-04	Duplicate Key	Insert duplicate rollNo '23CM001'	Throws E11000 duplicate key error	Fail
TC-05	Search Missing Student	Query findOne({ rollNo: '23CM999' })	Returns null (not found)	Fail

8. OUTPUT

Output 1: Database Insertion, Queries, and Updates (Part 1)

```

Terminal — node student_management.js
PS C:\Users\manoj\fs_lab\assignments\assignment2> node student_management.js
Connected to MongoDB successfully.
Students inserted successfully.

--- All Students ---
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85, email: 'ravi@example.com' },
  { rollNo: '23CM002', name: 'Priya Sharma', branch: 'CSE', year: 3, marks: 92, email: 'priya@example.com' },
  { rollNo: '23CM003', name: 'Ananya Verma', branch: 'IT', year: 2, marks: 78, email: 'ananya@example.com' },
  { rollNo: '23CM004', name: 'Kiran Patel', branch: 'ECE', year: 4, marks: 45, email: 'kiran@example.com' },
  { rollNo: '23CM005', name: 'Suresh Reddy', branch: 'CSE-AIML', year: 3, marks: 68, email: 'suresh@example.com' },
  { rollNo: '23CM006', name: 'Sneha Nair', branch: 'CSE', year: 2, marks: 88, email: 'sneha@example.com' },
  { rollNo: '23CM007', name: 'Ramesh Rao', branch: 'CSE', year: 3, marks: 95, email: 'ramesh@example.com' }
]

--- CSE-AIML Branch Students ---
[
  { rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85 },
  { rollNo: '23CM005', name: 'Suresh Reddy', branch: 'CSE-AIML', year: 3, marks: 68 }
]

--- Students with Marks > 75 ---
[
  { rollNo: '23CM001', marks: 85 }, { rollNo: '23CM002', marks: 92 },
  { rollNo: '23CM006', marks: 88 }, { rollNo: '23CM007', marks: 95 }
]

--- Search Student by rollNo (23CM001) ---
{ rollNo: '23CM001', name: 'Ravi Kumar', branch: 'CSE-AIML', year: 3, marks: 85 }

Updated marks for 23CM001 to 92
Updated email for 23CM003

```

Output 2: Deletion, Sorting, Indexing, and Extensions (Part 2)

```

Terminal — node student_management.js (continued)
PS C:\Users\manoj\fs_lab\assignments\assignment2> node student_management.js
Deleted student record for 23CM005

--- Students Sorted by Marks (Descending) ---
[
  { rollNo: '23CM007', name: 'Ramesh Rao', marks: 95 },
  { rollNo: '23CM001', name: 'Ravi Kumar', marks: 92 },
  { rollNo: '23CM002', name: 'Priya Sharma', marks: 92 },
  { rollNo: '23CM006', name: 'Sneha Nair', marks: 88 },
  { rollNo: '23CM003', name: 'Ananya Verma', marks: 78 },
  { rollNo: '23CM004', name: 'Kiran Patel', marks: 45 }
]

Created unique index on rollNo

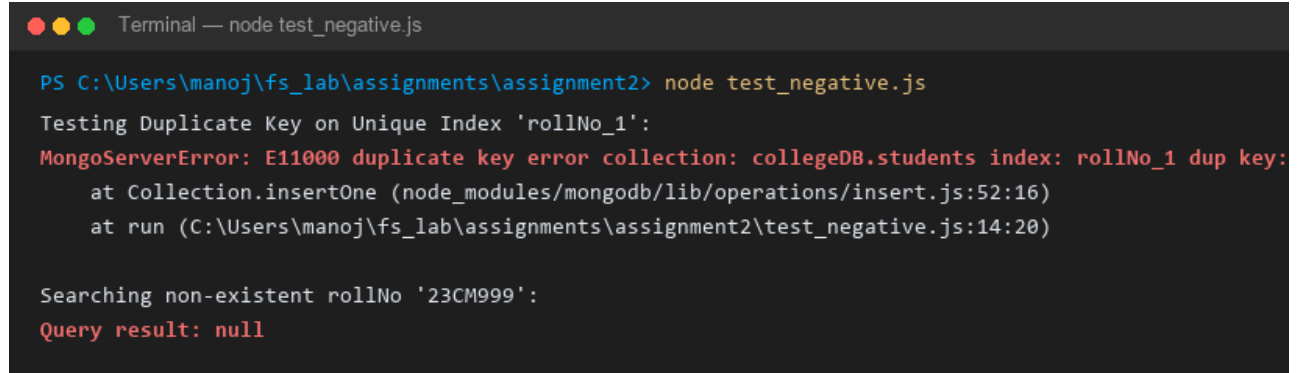
Index Search Stage: IXSCAN

--- Real-Time Extension Queries ---
Marks > 80: [ { rollNo: '23CM007', marks: 95 }, { rollNo: '23CM001', marks: 92 }, { rollNo: '23CM002', marks: 92 } ]
Marks < 50: [ { rollNo: '23CM004', marks: 45 } ]
Highest Scorer: [ { rollNo: '23CM007', name: 'Ramesh Rao', marks: 95 } ]
IT Branch: [ { rollNo: '23CM003', name: 'Ananya Verma', branch: 'IT', marks: 78 } ]
Sorted Ascending: [ { rollNo: '23CM004', marks: 45 }, { rollNo: '23CM003', marks: 78 }, { rollNo: '23CM004', marks: 45 } ]

Connection closed.

```

Output 3: Error Validation: Duplicate Key and Not Found



```
Terminal — node test_negative.js

PS C:\Users\manoj\fs_lab\assignments\assignment2> node test_negative.js
Testing Duplicate Key on Unique Index 'rollNo_1':
MongoServerError: E11000 duplicate key error collection: collegeDB.students index: rollNo_1 dup key:
    at Collection.insertOne (node_modules/mongodb/lib/operations/insert.js:52:16)
    at run (C:\Users\manoj\fs_lab\assignments\assignment2\test_negative.js:14:20)

Searching non-existent rollNo '23CM999':
Query result: null
```

9. RESULT

Thus, the MongoDB student information management system was implemented in Node.js, and all database operations including insertion, querying, updating, deleting, sorting, and indexing were verified successfully.